PIZZERÍA LA VIÑA

Manual de Usuario – Sitio Web

Elaborado por:

Sergio Gabriel Chaves Mosquera

SerStack – Desarrollo Web

Bogotá, D.C. – 2025

Ficha Técnica

A continuación se listan las tecnologías, herramientas y servicios utilizados en el desarrollo y despliegue del sitio web de Pizzería La Viña.

A. Frontend

Categoría	Tecnología / Valor
Framework	React 18 (Create React App)
Lenguaje	JavaScript (ES6+) + JSX
Enrutamiento	React Router DOM v6 — navegación multi-página (SPA)
Estilos	CSS Modules (un archivo .module.css por componente)
Tipografía	Google Fonts — Playfair Display + Lato
Íconos	React Icons
Animaciones	Framer Motion

B. Arquitectura de Datos

Categoría	Tecnología / Valor
Patrón	Centralización en src/data/menuData.js
Contenido	Menú, categorías, información del restaurante, horarios, contacto
Acceso	Importación directa desde cualquier componente React
Ventaja	Un solo lugar para editar precios, descripciones e imágenes

C. Infraestructura y Despliegue

Categoría	Tecnología / Valor
Control de versiones	Git + GitHub (repositorio privado)
Hosting / Despliegue	Vercel (Plataforma PaaS — tier gratuito)
Dominio	Subdominio Vercel gratuito (o dominio personalizado opcional)
CI/CD	Automático — cada push a main genera un nuevo deploy
Entorno local	Node.js 18+ + npm

D. Estructura del Proyecto

```
pizzeria-lavina/
├── public/
│   └── index.html            ← HTML base (no se toca)
├── src/
│   ├── data/
│   │   └── menuData.js       ← ⭐ Toda la info del restaurante
│   ├── pages/
│   │   ├── Home.jsx          ← Página principal
│   │   ├── Menu.jsx          ← Carta completa
│   │   ├── Nosotros.jsx      ← Historia / quiénes somos
│   │   └── Contacto.jsx      ← Formulario + mapa
│   ├── components/
│   │   ├── Navbar.jsx        ← Barra de navegación
│   │   ├── Footer.jsx        ← Pie de página
│   │   └── MenuCard.jsx      ← Tarjeta de cada plato
│   ├── App.jsx               ← Rutas principales (React Router)
│   └── index.js              ← Punto de entrada
├── package.json
└── README.md
```

Arquitectura y Funcionamiento

1. Arquitectura SPA con React Router

El sitio es una Single Page Application (SPA): el navegador descarga el proyecto una sola vez y React Router DOM se encarga de mostrar u ocultar páginas sin recargar. Esto da una experiencia muy rápida y fluida.

Las rutas definidas en App.jsx son:

- **/**Página principal (Home)
- **/menu**Carta del restaurante (Menu)
- **/nosotros**Historia y equipo (Nosotros)
- **/contacto**Formulario de contacto (Contacto)

> **Nota:** Aquí va el código actual de tu App.jsx con las rutas. Pégalo en este espacio o compártelo y lo incluyo directamente.

2. Centralización de Datos — menuData.js

Toda la información del restaurante vive en un único archivo: src/data/menuData.js. Si se necesita cambiar un precio, añadir un plato o actualizar el horario, solo hay que editar ese archivo y el cambio se refleja en toda la web automáticamente.

La estructura del archivo incluye:

- restaurantInfo — nombre, teléfono, dirección, horarios
- menuCategories — categorías del menú (pizzas, pastas, bebidas…)
- menuItems — cada plato con nombre, descripción, precio e imagen

> **Nota:** Pega aquí la estructura actual de tu menuData.js. Si quieres lo puedo regenerar contigo desde cero.

3. Componentes Reutilizables

React permite construir 'bloques' de interfaz que se reutilizan en distintas páginas. Los principales componentes del proyecto son:

Navbar.jsx

Barra de navegación fija en la parte superior. Contiene el logo de La Viña y los enlaces a las páginas. En móviles muestra un menú hamburguesa.

MenuCard.jsx

Tarjeta que muestra la imagen, nombre, descripción y precio de cada plato. Se usa dentro de Menu.jsx recorriendo el array menuItems con un .map().

Footer.jsx

Pie de página con dirección, redes sociales y enlace de WhatsApp directo.

> **Nota:** Si tienes más componentes creados (HeroSection, CategoryFilter, etc.), indícamelo y los añado aquí con su descripción.

4. Estilos con CSS Modules

Cada componente tiene su propio archivo de estilos .module.css. Esto evita que los estilos de un componente 'contaminen' a otro. Por ejemplo:

- **Navbar.module.css** solo aplica al Navbar
- **MenuCard.module.css** solo aplica a la tarjeta del menú

Para usar los estilos dentro del JSX se importan así:

```
import styles from './Navbar.module.css';

<nav className={styles.navbar}>...</nav>
```

Control de Versiones — Git y GitHub

¿Qué es Git?

Git es una herramienta que guarda el historial completo de cambios del código. Funciona como un "ctrl + Z" infinito y permite trabajar en paralelo sin perder versiones anteriores.

¿Qué es GitHub?

GitHub es la plataforma en la nube donde se aloja el repositorio del proyecto. Desde ahí, Vercel lee el código automáticamente para publicarlo en internet.

Estructura del Repositorio

El repositorio de Pizzería La Viña contiene:

- Rama main — versión en producción (la que ve el público)
- Commits — cada cambio guardado con un mensaje descriptivo
- .gitignore — archivos que Git ignora (node_modules, .env, etc.)

> **Nota:** Aquí puedes pegar la URL del repositorio de GitHub para incluirla en el manual, ej: github.com/tu-usuario/pizzeria-lavina

Comandos Esenciales

Para actualizar el sitio desde la terminal del computador:

1. Guardar cambios localmente

```
git add .
git commit -m "Descripción del cambio"
```

2. Subir cambios a GitHub (esto activa el deploy automático)

```
git push origin main
```

3. Ver historial de cambios

```
git log --oneline
```

> **Nota:** Cada vez que se hace git push, Vercel detecta el cambio y publica la nueva versión del sitio en aproximadamente 1-2 minutos.

Buenas Prácticas para los Commits

Los mensajes de commit deben ser descriptivos y cortos. Ejemplos recomendados:

- feat: agregar sección de ofertas en Home
- fix: corregir precio de la Pizza Margarita
- style: ajustar colores del navbar en móvil
- content: actualizar horarios de atención

Despliegue y Hosting — Vercel

¿Qué es Vercel?

Vercel es la plataforma donde vive el sitio web en internet. Es gratuita para proyectos personales y pequeños negocios, y está optimizada para proyectos React. Cada vez que se actualiza el código en GitHub, Vercel detecta el cambio y publica la nueva versión automáticamente.

Cómo Funciona el Deploy Automático (CI/CD)

El flujo completo desde que se edita el código hasta que el cliente ve el cambio en línea es:

1. Se edita el código en el computador (VS Code)
2. Se guarda el cambio con git add . && git commit -m "..."
3. Se sube a GitHub con git push origin main
4. Vercel detecta el push automáticamente (webhook)
5. Vercel ejecuta npm run build para compilar React
6. El sitio actualizado queda publicado en ~1-2 minutos

> **Nota:** No hace falta tocar el panel de Vercel para cada actualización. El proceso es completamente automático desde el momento en que se hace git push.

Panel de Control de Vercel

Desde el panel en vercel.com se puede:

- Ver el historial de todos los deploys realizados
- Revisar los logs de build si algo falla
- Hacer rollback a una versión anterior con un clic
- Configurar un dominio personalizado (ej: lavina.com.co)
- Ver métricas de visitas y rendimiento del sitio

Deploy de Vista Previa (Preview Deploys)

Vercel crea automáticamente una URL de vista previa única para cada rama o Pull Request de GitHub. Esto permite revisar cambios antes de publicarlos en el sitio principal sin afectar a los visitantes.

URL de producción: pizzeria-lavina.vercel.app (o dominio personalizado)

URL de preview: pizzeria-lavina-git-[rama]-[usuario].vercel.app

Variables de Entorno en Vercel

Si el proyecto usa claves secretas (API keys, tokens), estas se configuran en el panel de Vercel en Settings → Environment Variables. Nunca deben subirse al repositorio de GitHub.

> **Nota:** El proyecto actual de La Viña no requiere variables de entorno porque no consume APIs externas con credenciales. Esto podría cambiar si se integra un sistema de pedidos o pagos en el futuro.

Cómo Editar el Contenido del Sitio

Actualizar el Menú o Precios

Todo el contenido del menú está centralizado en src/data/menuData.js. Para cambiar un precio o descripción:

1. Abrir el archivo src/data/menuData.js en VS Code
2. Buscar el plato que se quiere editar (Ctrl + F para buscar)
3. Modificar el valor del campo (name, price, description, image)
4. Guardar el archivo (Ctrl + S)
5. Subir los cambios: git add . → git commit → git push

> **Nota:** Aquí pega el bloque de código actual de menuData.js (o uno de sus ítems) para que el cliente vea el formato exacto que debe seguir.

Agregar un Plato Nuevo

Para agregar un ítem al menú, se añade un objeto nuevo al array correspondiente en menuData.js con este formato:

```
{
  id: 15,
  category: 'pizzas',
  name: 'Pizza Hawaiana',
  description: 'Jamón, piña y mozzarella',
  price: 32000,
  image: '/images/hawaiana.jpg'
}
```

> **Nota:** El campo id debe ser único (un número que no exista ya en el array). El campo image puede ser una URL externa o una imagen guardada en la carpeta public/images/.

Actualizar Información de Contacto u Horarios

Los datos de contacto, dirección y horarios también viven en menuData.js, en el objeto restaurantInfo. Editar ese objeto es suficiente para que el cambio aparezca en todas las páginas donde se muestre esa información.

Correr el Proyecto en Local

Requisitos Previos

Para trabajar en el proyecto desde el computador se necesita tener instalado:

- Node.js 18 o superior — nodejs.org
- npm (viene incluido con Node.js)
- Git — git-scm.com
- VS Code (editor recomendado) — code.visualstudio.com

Instalación Inicial (solo una vez)

1. Clonar el repositorio

```
git clone https://github.com/[usuario]/pizzeria-lavina.git

cd pizzeria-lavina
```

2. Instalar dependencias

```
npm install
```

> **Nota:** La carpeta node_modules se genera automáticamente con npm install. Esta carpeta pesa varios cientos de MB y no se sube a GitHub (está en el .gitignore).

Iniciar el Servidor de Desarrollo

```
npm start
```

Esto abre el navegador en http://localhost:3000 con el sitio corriendo localmente. Los cambios se reflejan en tiempo real (hot reload) sin necesidad de recargar manualmente.

Compilar para Producción

```
npm run build
```

Este comando genera la carpeta /build con los archivos optimizados listos para publicar. Vercel lo ejecuta automáticamente en cada deploy — no es necesario correrlo manualmente.

Créditos y Contacto del Desarrollador

Desarrollador: Sergio Gabriel Chaves Mosquera
Agencia: SerStack — Desarrollo Web para Negocios
Instagram: @ser_stack
WhatsApp: +57 312 803 6725
Tecnologías: React · CSS Modules · React Router DOM · Vercel · GitHub

---

Documento elaborado en 2025 — SerStack